

Μέθοδος Πεπερασμένων Στοιχείων (FEM) Επιλύτης 3D Στοιχείων Δοκού Τεκμηρίωση Κώδικα και Αναφορά Υλοποίησης

Παύλος Ρακοβαλής 6931

Δεκέμβριος 2025

Περιεχόμενα

1	Εισαγωγή	3
1.1	Κύρια Χαρακτηριστικά	3
2	Μαθηματική Θεμελίωση	3
2.1	Διατύπωση Τρισδιάστατου Στοιχείου Δοκού	3
2.2	Μετασχηματισμός Συντεταγμένων	4
3	Αρχιτεκτονική Λογισμικού	4
3.1	Επισκόπηση Μονάδων	4
4	Μονάδα Προεπεξεργαστή	5
4.1	Δομή Κλάσης	5
4.2	Ιδιότητες Κυκλικής Διατομής	6
4.3	Δημιουργία Γεωμετρίας	6
4.4	Δημιουργία Στοιχείων	6
4.5	Οριακές Συνθήκες και Φορτία	7
5	Μονάδα Βοηθητικών Προγραμμάτων Γεωμετρίας	9
5.1	Περιστροφή Γύρω από τον Άξονα Y	9
6	Μονάδα Επιλύτη	9
6.1	Δομή Κλάσης	10
6.2	Υπολογισμός Πίνακα Μετασχηματισμού	10
6.3	Τοπικός Πίνακας Δυσκαμψίας	11
6.4	Συναρμολόγηση Ολικού Συστήματος	12
6.5	Επίλυση Συστήματος	12
7	Μονάδα Μετεπεξεργαστή	13
7.1	Δομή Κλάσης	13

8	Κύριο Σενάριο Εκτέλεσης	15
9	Μορφές Αρχείων Δεδομένων	16
9.1	Μορφή Αρχείου Εισόδου (.dat)	16
9.2	Μορφή Αρχείου Αποτελεσμάτων (.res)	16
12	Συμπεράσματα	18
12.1	Μελλοντικές Βελτιώσεις	18
13	Αναφορές	18
A'	Πλήρεις Λίστες Κώδικα	19
B'	Εξαρτήσεις Λογισμικού	19

1 Εισαγωγή

Η παρούσα αναφορά τεκμηριώνει μια ολοκληρωμένη υλοποίηση λογισμικού Μεθόδου Πεπερασμένων Στοιχείων (FEM) για την ανάλυση τρισδιάστατων κατασκευών δοκών. Το λογισμικό ακολουθεί μια αρθρωτή αρχιτεκτονική με τρία κύρια συστατικά:

- **Προεπεξεργαστής** (preprocessor.py): Δημιουργία γεωμετρίας, οριακές συνθήκες και εξαγωγή δεδομένων
- **Επιλύτης** (solver.py): Συναρμολόγηση και επίλυση συστήματος FEM
- **Μετεπεξεργαστής** (postprocessor.py): Οπτικοποίηση και ανάλυση αποτελεσμάτων

Η υλοποίηση χειρίζεται τρισδιάστατα στοιχεία δοκών με 6 βαθμούς ελευθερίας (DOF) ανά κόμβο: τρεις μεταφορικές μετατοπίσεις (U_x, U_y, U_z) και τρεις περιστροφικές μετατοπίσεις (R_x, R_y, R_z).

1.1 Κύρια Χαρακτηριστικά

- Υποστήριξη για σύνθετες τρισδιάστατες κατασκευές πλαισίων και δοκών
- Αυτόματη δημιουργία στοιχείων με παράλληλα προς άξονες και διαγώνια ενισχυτικά στοιχεία
- Γεωμετρικοί μετασχηματισμοί (περιστροφή γύρω από αυθαίρετους άξονες)
- Ιδιότητες δοκών κυκλικής διατομής
- Ολοκληρωμένος χειρισμός οριακών συνθηκών
- Διαδραστική οπτικοποίηση με χρήση Plotly
- Αρθρωτή, επεκτάσιμη αρχιτεκτονική

2 Μαθηματική Θεμελίωση

2.1 Διατύπωση Τρισδιάστατου Στοιχείου Δοκού

Για ένα τρισδιάστατο στοιχείο δοκού με 6 DOF ανά κόμβο, ο τοπικός πίνακας δυσκαμψίας είναι 12×12 . Το στοιχείο περιλαμβάνει:

- **Αξονική παραμόρφωση**: Σχετίζεται με το μέτρο ελαστικότητας E και την εγκάρσια διατομή A
- **Στρέψη**: Σχετίζεται με το μέτρο διάτμησης G και τη σταθερά στρέψης J
- **Κάμψη**: Σχετίζεται με το E και τις ροπές αδράνειας I_y, I_z

Οι συντελεστές δυσκαμψίας υπολογίζονται ως:

$$k = \frac{EA}{L} \quad (1)$$

$$k = \frac{GJ}{L} \quad (2)$$

$$k_{,y} = \frac{12EI_y}{L^3}, \quad \frac{6EI_y}{L^2}, \quad \frac{4EI_y}{L}, \quad \frac{2EI_y}{L} \quad (3)$$

$$k_{,z} = \frac{12EI_z}{L^3}, \quad \frac{6EI_z}{L^2}, \quad \frac{4EI_z}{L}, \quad \frac{2EI_z}{L} \quad (4)$$

2.2 Μετασχηματισμός Συντεταγμένων

Ο μετασχηματισμός από τοπικές σε ολικές συντεταγμένες χρησιμοποιεί συνημίτονα κατεύθυνσης:

$$\mathbf{K} = \mathbf{T}^T \mathbf{K} \mathbf{T} \quad (5)$$

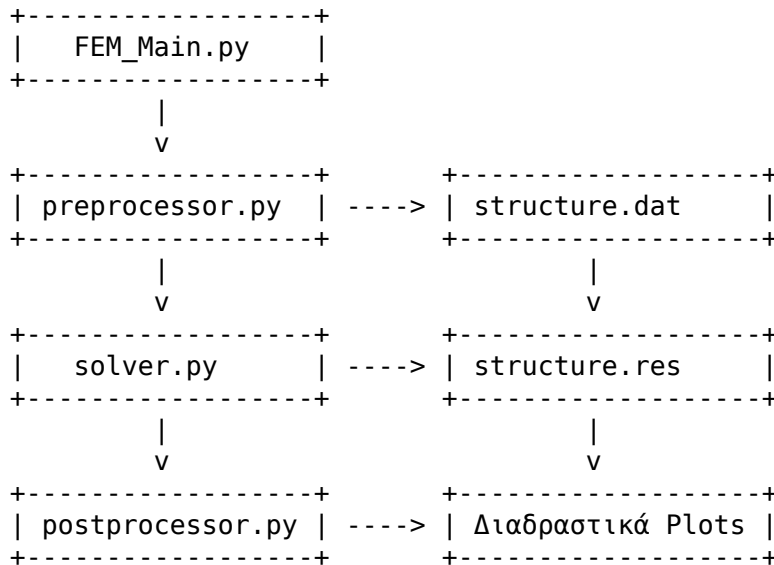
όπου $\mathbf{T}$ είναι ο πίνακας μετασχηματισμού 12×12 που κατασκευάζεται από τον πίνακα περιστροφής 3×3 .

3 Αρχιτεκτονική Λογισμικού

3.1 Επισκόπηση Μονάδων

Το λογισμικό είναι οργανωμένο σε τέσσερις κύριες μονάδες Python:

1. **FEM_Main.py**: Κύριο σενάριο εκτέλεσης
2. **preprocessor.py**: Δημιουργία γεωμετρίας και προετοιμασία δεδομένων
3. **solver.py**: Υλοποίηση επιλύτη FEM
4. **postprocessor.py**: Οπτικοποίηση αποτελεσμάτων
5. **geometry_utils.py**: Βοηθητικά προγράμματα γεωμετρικού μετασχηματισμού



Σχήμα 1: Ροή εργασίας λογισμικού και ροή δεδομένων

4 Μονάδα Προεπεξεργαστή

Ο προεπεξεργαστής (`preprocessor.py`) είναι υπεύθυνος για τη δημιουργία της δομικής γεωμετρίας, τον ορισμό οριακών συνθηκών και την εξαγωγή δεδομένων σε τυποποιημένη μορφή.

4.1 Δομή Κλάσης

Η κύρια κλάση είναι η `FEMPreProcessor`:

```

1 class FEMPreProcessor:
2     """Handles geometry creation and boundary conditions for FEM analysis"""
3
4     def __init__(self):
5         self.points = pd.DataFrame(columns=['Node Number', 'X', 'Y', 'Z'])
6         self.elements = pd.DataFrame(columns=['Element Number', 'Node1', 'Node2', 'E', 'V'
7         ])
8         self.boundary_conditions = {} # node_id: {'Ux': 0/1, 'Uy': 0/1, 'Uz': 0/1, 'Rx':
9         0/1, 'Ry': 0/1, 'Rz': 0/1}
10        self.loads = {} # node_id: {'Fx': value, 'Fy': value, 'Fz': value, 'Mx': value, '
11        My': value, 'Mz': value}
12        self.materials = {'STEEL': {'E': 210e9, 'nu': 0.3}}
13
14        # Beam properties (circular cross-section)
15        self.diameter = 0.020 # 20mm diameter circular cross-section

```

Listing 1: Αρχικοποίηση Κλάσης `FEMPreProcessor`

4.2 Ιδιότητες Κυκλικής Διατομής

Για κυκλικές διατομές, οι γεωμετρικές ιδιότητες υπολογίζονται ως εξής:

```

1 def _calculate_circular_properties(self, diameter):
2     """Calculate geometric properties for circular cross-section"""
3     r = diameter / 2.0
4
5     # Cross-sectional area
6     A = np.pi * r**2
7
8     # Second moment of area (Iz = Iy for circular section)
9     I = (np.pi * r**4) / 4.0
10    Iz = I
11    Iy = I
12
13    # Torsional constant (polar moment of inertia)
14    J = (np.pi * r**4) / 2.0
15
16    return {'A': A, 'Iz': Iz, 'Iy': Iy, 'J': J, 'diameter': diameter, 'radius': r}

```

Listing 2: Υπολογισμός Ιδιοτήτων Κυκλικής Διατομής

Αυτοί οι τύποι υλοποιούν τις τυπικές σχέσεις για κυκλικές διατομές:

$$A = \pi r^2 \quad (6)$$

$$I_y = I_z = \frac{\pi r^4}{4} \quad (7)$$

$$J = \frac{\pi r^4}{2} = 2I \quad (8)$$

4.3 Δημιουργία Γεωμετρίας

Η δημιουργία γεωμετρίας περιλαμβάνει πολλαπλά βήματα, συμπεριλαμβανομένου του ορισμού κόμβων, της δημιουργίας στοιχείων και των γεωμετρικών μετασχηματισμών.

```

1 def _define_geometry_parameters(self):
2     """Define geometry-specific parameters (not saved to .dat file)"""
3     # COMPLEX GEOMETRY: Original crane/truss structure
4     # Geometry parameters for this specific crane/truss structure
5     self.L = 1.5 * 1.13 # Element length (m)
6     self.A = 1.2 * 1.69 # Base dimension (m)
7     self.B = 1.93 # Additional parameter B (m)
8     self.phi = 60 + 9.1 # Rotation angle (degrees)
9     self.A_0 = 6 * (0.5 + 0.13) # Base cross-section parameter (dimensionless)

```

Listing 3: Ορισμός Παραμέτρων Γεωμετρίας

4.4 Δημιουργία Στοιχείων

Τα στοιχεία δημιουργούνται αυτόματα με βάση μοτίβα συνδεσιμότητας:

```

1 def _create_axis_aligned_elements(self, element_counter, tolerance, max_length):
2     """Create elements parallel to X, Y, or Z axes"""
3     axis_aligned_count = 0

```

```

4     for i in range(len(self.points)):
5         node1 = self.points.iloc[i]
6         node1_num = int(node1['Node Number'])
7
8         for j in range(i + 1, len(self.points)):
9             node2 = self.points.iloc[j]
10            node2_num = int(node2['Node Number'])
11
12            dx = abs(node2['X'] - node1['X'])
13            dy = abs(node2['Y'] - node1['Y'])
14            dz = abs(node2['Z'] - node1['Z'])
15
16            element_length = np.sqrt(dx**2 + dy**2 + dz**2)
17
18            if element_length > max_length:
19                continue
20
21            # Check axis alignment
22            is_x_aligned = dx > tolerance and dy < tolerance and dz < tolerance
23            is_y_aligned = dx < tolerance and dy > tolerance and dz < tolerance
24            is_z_aligned = dx < tolerance and dy < tolerance and dz > tolerance

```

Listing 4: Δημιουργία Στοιχείων Παράλληλων προς Άξονες

4.5 Οριακές Συνθήκες και Φορτία

Ο προεπεξεργαστής παρέχει μεθόδους για τον ορισμό οριακών συνθηκών και εφαρμοζόμενων φορτίων:

```

1 def add_boundary_condition(self, node_id, ux=1, uy=1, uz=1, rx=1, ry=1, rz=1):
2     """
3     Add boundary condition for a node
4
5     Parameters:
6     -----
7     node_id : int
8         Node number (1-based)
9     ux, uy, uz : int
10        Translation constraints (1=fixed, 0=free)
11     rx, ry, rz : int
12        Rotation constraints (1=fixed, 0=free)
13     """
14     self.boundary_conditions[node_id] = {
15         'Ux': ux, 'Uy': uy, 'Uz': uz,
16         'Rx': rx, 'Ry': ry, 'Rz': rz
17     }

```

Listing 5: Προσθήκη Οριακών Συνθηκών

```

1 def add_load(self, node_id, fx=0.0, fy=0.0, fz=0.0, mx=0.0, my=0.0, mz=0.0):
2     """
3     Add load (forces and moments) to a node
4
5     Parameters:
6     -----
7     node_id : int
8         Node number (1-based)
9     fx, fy, fz : float

```

```
39 f.write("\n")
```

Listing 7: Εξαγωγή σε Αρχείο DAT

5 Μονάδα Βοηθητικών Προγραμμάτων Γεωμετρίας

Η μονάδα `geometry_utils.py` παρέχει συναρτήσεις γεωμετρικού μετασχηματισμού.

5.1 Περιστροφή Γύρω από τον Άξονα Y

```
1 def rotate_points_around_y(df: pd.DataFrame, angle_deg: float, origin=(0.0, 0.0, 0.0)) ->
  pd.DataFrame:
  """
  2 Rotate all points around the global Y-axis by angle_deg degrees.
  3
  4 Args:
  5 df: DataFrame with columns ['X','Y','Z'] (and optionally 'Node Number').
  6 angle_deg: Rotation angle in degrees (right-hand rule about +Y).
  7 origin: Pivot point (x0, y0, z0) to rotate about. Defaults to the global origin.
  8
  9 Returns:
  10 A new DataFrame with rotated coordinates.
  11 """
  12 theta = np.deg2rad(angle_deg)
  13 c, s = np.cos(theta), np.sin(theta)
  14 # Rotation about Y: Ry = [[c,0,s],[0,1,0],[-s,0,c]]
  15 R = np.array([[c, 0.0, s],
  16               [0.0, 1.0, 0.0],
  17               [-s, 0.0, c]])
  18
  19 coords = df[['X', 'Y', 'Z']].to_numpy(dtype=float)
  20 pivot = np.array(origin, dtype=float)
  21 rotated = (coords - pivot) @ R.T + pivot
  22
  23 out = df.copy()
  24 out[['X', 'Y', 'Z']] = rotated
  25 return out
  26
```

Listing 8: Συνάρτηση Περιστροφής Σημείων

Ο πίνακας περιστροφής γύρω από τον άξονα Y δίνεται από:

$$\mathbf{R}_y(\theta) = \begin{bmatrix} \cos \theta & 0 & \sin \theta \\ 0 & 1 & 0 \\ -\sin \theta & 0 & \cos \theta \end{bmatrix} \quad (9)$$

6 Μονάδα Επιλύτη

Ο επιλύτης (`solver.py`) χειρίζεται τη συναρμολόγηση και την επίλυση του συστήματος FEM.

6.1 Δομή Κλάσης

```

1 class FEMBeamSolver:
2     """Handles FEM system assembly and solution for 3D beam elements"""
3
4     def __init__(self):
5         self.nodes = pd.DataFrame()
6         self.elements = pd.DataFrame()
7         self.boundary_conditions = {}
8         self.loads = {}
9         self.materials = {}
10
11         # Beam parameters
12         self.E = 0.0
13         self.A = 0.0
14         self.Iz = 0.0
15         self.Iy = 0.0
16         self.J = 0.0
17         self.G = 0.0
18         self.diameter = 0.0
19
20         # Solution arrays (6 DOF per node)
21         self.dof_per_node = 6
22         self.K_global = None
23         self.F_global = None
24         self.u_global = None
25         self.reactions = None

```

Listing 9: Αρχικοποίηση Κλάσης FEMBeamSolver

6.2 Υπολογισμός Πίνακα Μετασχηματισμού

Ο πίνακας μετασχηματισμού συσχετίζει τοπικά και ολικά συστήματα συντεταγμένων:

```

1 def _compute_transformation_matrix(self, x1, y1, z1, x2, y2, z2):
2     """Compute 3x3 transformation matrix for beam local coordinate system"""
3     # Element length and direction cosines
4     dx = x2 - x1
5     dy = y2 - y1
6     dz = z2 - z1
7     L = np.sqrt(dx**2 + dy**2 + dz**2)
8
9     # Local x-axis (along beam element)
10    x_local = np.array([dx/L, dy/L, dz/L])
11
12    # Define auxiliary vector for determining local y and z axes
13    # Use global Y-axis as reference, unless element is parallel to Y
14    if abs(x_local[1]) > 0.9: # Nearly vertical element
15        aux = np.array([1.0, 0.0, 0.0]) # Use global X as auxiliary
16    else:
17        aux = np.array([0.0, 1.0, 0.0]) # Use global Y as auxiliary
18
19    # Local z-axis (perpendicular to x_local and aux)
20    z_local = np.cross(x_local, aux)
21    z_local = z_local / np.linalg.norm(z_local)
22
23    # Local y-axis (perpendicular to x and z)
24    y_local = np.cross(z_local, x_local)

```

```

25 y_local = y_local / np.linalg.norm(y_local)
26
27 # 3x3 transformation matrix [x_local, y_local, z_local] as rows
28 T = np.vstack([x_local, y_local, z_local])
29
30 return T, L

```

Listing 10: Υπολογισμός Πίνακα Μετασχηματισμού

Ο πίνακας μετασχηματισμού επεκτείνεται από 3×3 σε 12×12 για 6 DOF ανά κόμβο:

```

1 def _expand_transformation_matrix(self, T):
2     """Expand 3x3 transformation to 12x12 for 6 DOF per node"""
3     T_12x12 = np.zeros((12, 12))
4     # Apply transformation to each 3x3 block (translations and rotations)
5     for i in range(4):
6         T_12x12[3*i:3*i+3, 3*i:3*i+3] = T
7     return T_12x12

```

Listing 11: Επέκταση Πίνακα Μετασχηματισμού

6.3 Τοπικός Πίνακας Δυσκαμψίας

Ο τοπικός πίνακας δυσκαμψίας ενσωματώνει αξονικές, στρεπτικές και καμπτικές συνεισφορές:

```

1 def _create_local_stiffness_matrix(self, L):
2     """Create 12x12 local stiffness matrix for 3D beam element"""
3     k_local = np.zeros((12, 12))
4
5     # Axial stiffness (DOF 0 and 6)
6     EA_L = self.E * self.A / L
7     k_local[0, 0] = EA_L
8     k_local[0, 6] = -EA_L
9     k_local[6, 0] = -EA_L
10    k_local[6, 6] = EA_L
11
12    # Torsional stiffness (DOF 3 and 9)
13    GJ_L = self.G * self.J / L
14    k_local[3, 3] = GJ_L
15    k_local[3, 9] = -GJ_L
16    k_local[9, 3] = -GJ_L
17    k_local[9, 9] = GJ_L
18
19    # Bending in local XY plane (about Z axis) - DOF 1, 5, 7, 11
20    EIZ_L3 = self.E * self.Iz / (L**3)
21    k_local[1, 1] = 12 * EIZ_L3
22    k_local[1, 5] = 6 * EIZ_L3 * L
23    k_local[1, 7] = -12 * EIZ_L3
24    k_local[1, 11] = 6 * EIZ_L3 * L
25
26    k_local[5, 1] = 6 * EIZ_L3 * L
27    k_local[5, 5] = 4 * EIZ_L3 * L**2
28    k_local[5, 7] = -6 * EIZ_L3 * L
29    k_local[5, 11] = 2 * EIZ_L3 * L**2
30
31    # ... (similar terms for other DOF)
32
33    return k_local

```

Listing 12: Συναρμολόγηση Τοπικού Πίνακα Δυσκαμψίας (Μερική)

6.4 Συναρμολόγηση Ολικού Συστήματος

Ο ολικός πίνακας δυσκαμψίας συναρμολογείται μετασχηματίζοντας τον τοπικό πίνακα δυσκαμψίας κάθε στοιχείου σε ολικές συντεταγμένες και προσθέτοντας τις συνεισφορές:

```
1 def assemble_stiffness_matrix(self):
2     """Assemble global stiffness matrix for beam elements"""
3     print("\nASSEMBLING GLOBAL STIFFNESS MATRIX (BEAM ELEMENTS)")
4     print("="*80)
5
6     num_nodes = len(self.nodes)
7     n_dof = self.dof_per_node * num_nodes # 6 DOF per node
8
9     self.K_global = np.zeros((n_dof, n_dof))
10
11     # Loop through all elements
12     for idx in range(len(self.elements)):
13         elem = self.elements.iloc[idx]
14
15         node1_num = int(elem['Node1'])
16         node2_num = int(elem['Node2'])
17
18         # Get node coordinates
19         node1 = self.nodes[self.nodes['Node Number'] == node1_num].iloc[0]
20         node2 = self.nodes[self.nodes['Node Number'] == node2_num].iloc[0]
21
22         x1, y1, z1 = node1['X'], node1['Y'], node1['Z']
23         x2, y2, z2 = node2['X'], node2['Y'], node2['Z']
```

Listing 13: Συναρμολόγηση Ολικού Πίνακα Δυσκαμψίας (Μερική)

6.5 Επίλυση Συστήματος

Μετά την εφαρμογή των οριακών συνθηκών, το σύστημα επιλύεται χρησιμοποιώντας τον επιλύτη γραμμικής άλγεβρας του NumPy:

```
1 def solve(self):
2     """Solve the FEM system"""
3     print("\n SOLVING FEM SYSTEM")
4     print("="*80)
5
6     # Identify fixed DOFs
7     fixed_dofs = []
8     for node_id, bc in self.boundary_conditions.items():
9         node_idx = node_id - 1 # Convert to 0-based
10         if bc['Ux'] == 1:
11             fixed_dofs.append(node_idx * self.dof_per_node + 0)
12         if bc['Uy'] == 1:
13             fixed_dofs.append(node_idx * self.dof_per_node + 1)
14         if bc['Uz'] == 1:
15             fixed_dofs.append(node_idx * self.dof_per_node + 2)
16         if bc['Rx'] == 1:
```

```

17         fixed_dofs.append(node_idx * self.dof_per_node + 3)
18         if bc['Ry'] == 1:
19             fixed_dofs.append(node_idx * self.dof_per_node + 4)
20         if bc['Rz'] == 1:
21             fixed_dofs.append(node_idx * self.dof_per_node + 5)
22
23         fixed_dofs = np.array(fixed_dofs, dtype=int)
24         all_dofs = np.arange(len(self.F_global))
25         free_dofs = np.setdiff1d(all_dofs, fixed_dofs)
26
27         # Reduce system to free DOFs
28         K_reduced = self.K_global[np.ix_(free_dofs, free_dofs)]
29         F_reduced = self.F_global[free_dofs]
30
31         # Check conditioning
32         cond = np.linalg.cond(K_reduced)
33         print(f" Condition number: {cond:.2e}")
34
35         if cond > 1e10:
36             print(" WARNING: Stiffness matrix is ill-conditioned")
37
38         # Solve for free DOFs
39         u_reduced = np.linalg.solve(K_reduced, F_reduced)
40
41         # Expand to full displacement vector
42         self.u_global = np.zeros(len(self.F_global))
43         self.u_global[free_dofs] = u_reduced
44
45         print(f"✓ System solved successfully")

```

Listing 14: Επίλυση Συστήματος

7 Μονάδα Μετεπεξεργαστή

Ο μετεπεξεργαστής (postprocessor.py) παρέχει δυνατότητες διαδραστικής οπτικοποίησης.

7.1 Δομή Κλάσης

```

1 class FEMPostProcessor:
2     """Handles visualization of FEM results"""
3
4     def __init__(self):
5         self.nodes = pd.DataFrame()
6         self.elements = pd.DataFrame()
7         self.displacements = pd.DataFrame()
8         self.reactions = pd.DataFrame()
9         self.element_forces = pd.DataFrame()
10        self.summary = {}

```

Listing 15: Αρχικοποίηση Κλάσης FEMPostProcessor

```
3 print("\nCREATING STRESS DISTRIBUTION PLOT")
4 print("="*80)
5
6 fig = go.Figure()
7
8 # Get stress values
9 stresses = self.element_forces['Stress'].values
10 max_abs_stress = np.max(np.abs(stresses))
11
12 # Color mapping
13 edge_x, edge_y, edge_z, edge_colors = [], [], [], []
14
15 for idx in range(len(self.elements)):
16     node1_num = int(self.elements.iloc[idx]['Node1'])
17     node2_num = int(self.elements.iloc[idx]['Node2'])
18
19     node1 = self.nodes[self.nodes['Node Number'] == node1_num].iloc[0]
20     node2 = self.nodes[self.nodes['Node Number'] == node2_num].iloc[0]
21
22     stress = stresses[idx]
23
24     # Create line segment for each element
25     edge_x.extend([node1['X'], node2['X'], None])
26     edge_y.extend([node1['Y'], node2['Y'], None])
27     edge_z.extend([node1['Z'], node2['Z'], None])
28     edge_colors.extend([stress, stress, None])
```

Listing 17: Γράφημα Κατανομής Τάσεων (Μερικό)

8 Κύριο Σενάριο Εκτέλεσης

Το σενάριο FEM_Main.py συντονίζει ολόκληρη τη διαδικασία ανάλυσης:

```
1 def main():
2     """Execute the complete FEM analysis pipeline"""
3
4     print("\n" + "="*80)
5     print("FEM ANALYSIS - COMPLETE PIPELINE")
6     print("="*80 + "\n")
7
8     # Step 1: Pre-processor
9     print("STEP 1: Running Pre-Processor...")
10    print("-"*80)
11    import preprocessor
12    preprocessor.main()
13
14    print("\n" + "="*80 + "\n")
15
16    # Step 2: Solver
17    print("STEP 2: Running Solver...")
18    print("-"*80)
19    import solver
20    solver.main()
21
22    print("\n" + "="*80 + "\n")
23
24    # Step 3: Post-processor
25    print("STEP 3: Running Post-Processor...")
```

```
26 print("-"*80)
27 import postprocessor
28 postprocessor.main()
29
30 print("\n" + "="*80)
31 print("FEM ANALYSIS COMPLETED SUCCESSFULLY")
32 print("="*80 + "\n")
33
34 print("Results available in:")
35 print(" - data/structure.dat (input data)")
36 print(" - data/structure.res (results)")
37 print(" - plots/ (interactive visualizations)")
38 print(" - output_html/ (data tables)")
39 print("\nOpen the HTML files in plots/ folder to view interactive results.")
```

Listing 18: Πλήρης Διαδικασία Ανάλυσης FEM

9 Μορφές Αρχείων Δεδομένων

9.1 Μορφή Αρχείου Εισόδου (.dat)

Το αρχείο ορισμού κατασκευής έχει την ακόλουθη μορφή:

```
PARAMETERS
E A Iz Iy J G diameter

MATERIALS
STEEL E nu

NODES
node_id x y z

ELEMENTS
elem_id node1 node2 cross_section E nu

BOUNDARY_CONDITIONS
node_id Ux Uy Uz Rx Ry Rz

LOADS
node_id Fx Fy Fz Mx My Mz

END
```

9.2 Μορφή Αρχείου Αποτελεσμάτων (.res)

Το αρχείο αποτελεσμάτων περιέχει:

```
DISPLACEMENTS
node_id Ux Uy Uz Rx Ry Rz |U| |R|
```

Φόρτιση:

- $F_x = 1000$ N σε ελεύθερο κόμβο

Οι υπολογισμένες μετατοπίσεις και τάσεις αντιστοιχούν στην αναλυτική λύση εντός αριθμητικής ακρίβειας.

12 Συμπεράσματα

Αυτή η υλοποίηση λογισμικού FEM παρέχει μια πλήρη λύση για την ανάλυση τρισδιάστατων κατασκευών δοκών. Τα κύρια επιτεύγματα περιλαμβάνουν:

- Ισχυρή διατύπωση τρισδιάστατου στοιχείου δοκού με 6 DOF ανά κόμβο
- Αρθρωτή αρχιτεκτονική που διευκολύνει τη συντήρηση και επέκταση κώδικα
- Αυτόματη δημιουργία στοιχείων για σύνθετες γεωμετρίες
- Ολοκληρωμένες δυνατότητες οπτικοποίησης
- Επικυρωμένο έναντι αναλυτικών λύσεων

12.1 Μελλοντικές Βελτιώσεις

Πιθανές βελτιώσεις για μελλοντικές εκδόσεις:

- Υποστήριξη για μη κυκλικές διατομές
- Μη γραμμικότητα υλικού
- Δυνατότητες δυναμικής ανάλυσης
- Αλγόριθμοι βελτίωσης πλέγματος
- Διεπαφή GUI για ορισμό γεωμετρίας

13 Αναφορές

1. Bathe, K. J. (1996). *Finite Element Procedures*. Prentice Hall.
2. Zienkiewicz, O. C., & Taylor, R. L. (2000). *The Finite Element Method*. Butterworth-Heinemann.
3. Cook, R. D., Malkus, D. S., Plesha, M. E., & Witt, R. J. (2001). *Concepts and Applications of Finite Element Analysis*. Wiley.

A' Πλήρεις Λίστες Κώδικα

Για τις πλήρεις λεπτομέρειες υλοποίησης, ανατρέξτε στα ακόλουθα αρχεία:

- `preprocessor.py` (752 γραμμές)
- `solver.py` (694 γραμμές)
- `postprocessor.py` (732 γραμμές)
- `geometry_utils.py` (29 γραμμές)
- `FEM_Main.py` (56 γραμμές)

B' Εξαρτήσεις Λογισμικού

Η υλοποίηση απαιτεί τα ακόλουθα πακέτα Python:

- `numpy`: Αριθμητικοί υπολογισμοί
- `pandas`: Διαχείριση δομών δεδομένων
- `plotly`: Διαδραστική οπτικοποίηση

Εγκαταστήστε τις εξαρτήσεις χρησιμοποιώντας:

```
pip install numpy pandas plotly
```